

Performance Metrics

01) Gaussian Elimination using OpenMP

1.a) Sequential Gaussian elimination

⇒ Algorithm

1. Forward Elimination

for every column k :

1. Find the row having the largest absolute value in column k .

2. Swap that row with row k .

This is partial pivoting.

3. For every row below the pivot:

$$\text{factor} = \frac{A[i][k]}{A[k][k]}$$

4. Make:

$$A[i][k] = 0$$

5. Update the remaining elements:

$$A[i][j] = A[i][j] - \text{factor} \times A[k][j]$$

6. Update:

$$b[i] = b[i] - \text{factor} \times b[k]$$

Complexity

Gaussian elimination

$$O(N^3)$$

b) Parallel version

⇒ Algorithm

1. Read A, b ; set $P = \text{omp_set_num_threads}(P)$

2. $\text{start} = \text{omp_get_wtime}()$

3. For $k = 0$ to $N-2$:

a. Find pivot row (sequential scan, or parallel max-reduction on $|A(i)[k]|$ using a custom reduction / critical section to store both value & index)

b. Swap rows if needed (single thread)

c. `#pragma omp parallel for schedule('static') private(factor, j)`

• For $j = k+1$ to $N-1$:

~~$A[i][j] = \text{factor} * A[k][j]$~~

~~record(row i handled by thread tid)~~

~~$\text{tid} = \text{omp_get_thread_num}()$~~

~~$\text{factor} = A[i][k] / A[k][k]$~~

• for $j = k$ to N :

~~$A[i][j] = \text{factor} * A[k][j]$~~

~~record(row i handled by thread tid)~~

~~// implicit barrier at end of parallel for before next k~~

4. BACKWARD SUBSTITUTION (inherently sequential over i ; inner sum can be parallel):

for $i = N-1$ down to 0 :

$\text{sum} = A[i][N]$

`#pragma omp parallel for reduction(-:sum)`

for $j = i+1$ to $N-1$: $\text{sum} += A[i][j] * x[j]$

$x[i] = \text{sum} / A[i][i]$

5. $\text{end} = \text{omp_get_wtime}()$; $T(P) = \text{end} - \text{start}$

6. compare $x[]$ against serial result from (a); print thread-for-

c) Measure $T(P)$ for $N = 2000$, $P = 1, 2, 4, 8$

$\Rightarrow$ 1. Generate a fixed/random diagonally-dominant $N \times N$ system ($N = 2000$) once, reuse for all P .

2. for each P in $\{1, 2, 4, 8\}$:

a. $\text{omp_set_num_threads}(P)$

b. $\text{start} = \text{omp_get_wtm}()$

c. Run parallel Gauss-Jordan elimination + backward substitution from (b)

d. $\text{end} = \text{omp_get_wtm}()$

e. $T(P) = \text{end} - \text{start}$

• Result:

P	$T(P)$ sec
1	1.7008
2	1.1062
4	0.8906
8	0.8134

d) Speedup & Efficiency

$\Rightarrow$ 1. $T(1) = \text{serial time for } N=2000$

2. for P in $\{2, 4, 8\}$:

$$S(P) = T(1) / T(P)$$

$$E(P) = S(P) / P$$

3. classify each P :

if $S(P) = P$

$\rightarrow$ Linear

$\rightarrow$ Sub-linear

if $S(P) < P$

$\rightarrow$ Super-linear

if $S(P) > P$

$\rightarrow$ Super-linear

4. Scan P in increasing order

report first P where $E(P) < 0.8$

5. Print table

$P \mid T(P) \mid E(P)$

classification

Q2) Odd - Even Transposition Sort

a) Sequential

=> 1. Input array $A[0 \dots N-1] = \{29, 10, 14, 37, 13, 5, 40, 8, 21, 3\}$

2. $N = 10$
 $start = omp_get_wtme()$

3. for phase = 0 to $N-1$:

if phase is even:

for $i = 0, 2, 4, \dots, (i < N-1)$:

if $A[i] > A[i+1]$: swap($A[i]$, $A[i+1]$)

else:

for $i = 1, 3, 5, \dots, (i < N-1)$:

if $A[i] > A[i+1]$: swap($A[i]$, $A[i+1]$)

4. $end = omp_get_wtme()$; $T1 = end - start$

5. Verify: for $i = 0$ to $N-2$: assert $A[i] \leq A[i+1]$

6. Print sorted array, $T1$

(b) Parallel

1. Input array $A[0 \dots N-1]$: first verify correctness with the

$N = 10$ test case from (a)

2. Then generate random array with $N = 10,000$

3. for each P in $\{1, 2, 4, 8\}$:

a. $omp_set_num_threads(P)$

b. $start = omp_get_wtme()$

c. for phase = 0 to $N-1$:

if phase is even:

#pragma omp parallel for schedule(static)

for $i = 0$; $i < N-1$; $i += 2$: if $A[i] > A[i+1]$:

swap($A[i]$, $A[i+1]$)

else:

if pragma omp parallel for schedule (static)

for (i = 1; i < N-1; i++)

if (A[i] > A[i+1])

swap(A[i], A[i+1])

d. end = omp_get_wtime(); T(P) = end - start

4. verify sorted correctness against a reference sort.

c) Speedup and Efficiency

1. T(1) from (b) with P=1 as baseline

2. for P in {2, 4, 8}:

$$S(P) = T(1) / T(P)$$

$$E(P) = S(P) / P$$

3. Classify linear / sub-linear / super-linear as in Q1(d)

4. Report first P where $E(P) < 0.5$

5. Tabulate: P | T(P) | S(P) | E(P) | classification.

d) Amdahl vs Gustafson

1. Take measured S(8) from part (c)

2. Solve inverse Amdahl's Law for parallel fraction f:

$$S(P) = 1 / [(1-f) + f/P]$$

$$\Rightarrow f = (1 - 1/S(8)) / (1 - 1/8)$$

3. Amdahl speedup prediction (fixed prob size):

$$S_{\text{Amdahl}}(8) = 1 / [(1-f) + f/8] \quad (\text{sanity check} = \text{measured } S(8))$$

4. Gustafson scaled speedup (prob size scales with P), using

$$S_{\text{Gustafson}}(8) = 8 - S^*(8-1) = 8 - (1-f) * 7 \quad S = 1-f$$

5. Compare S-Andahl (8) vs S-Crustafson (8)

Since N is fixed in the exp, Andahl's fixed-path size assumption matches the setup more closely than Crustafson's scaled-problem-size approach.